

TD Apprentissage Automatique : *Feuille de Triche Python*

Numpy

```
# Importer numpy
import numpy as np
# Créer une liste
liste = np.array([1, 2, 3])
# Créer une liste de zeros
liste = np.zeros(nombre)
# Forme d'une matrice
taille = liste.shape
# Taille de la liste (Prem. dim. de la matr.)
taille = liste.shape[0]
# Trier les indices une liste
liste_tri_indices = np.argsort(liste)
# Valeur minimale d'une liste
max_liste = np.max(liste)
# Indice de la valeur maximale d'une liste
ind_max_liste = np.argmax(liste)
```

Math

```
# Importer math
import math
# Racine carrée
res = math.sqrt(valeur)
```

Chargement des données

```
# Importer iris
from sklearn.datasets import load_iris
# Charger IRIS (pour description)
dataset = load_iris()
# Charger Breast Cancer (pour description)
dataset = load_breast_cancer()
# Description du jeu
print(dataset.DESCR)
# Charger IRIS (pour manipulation)
X, y = load_iris(return_X_y=True)
# Charger Breast Cancer (pour manipulation)
X, y = load_breast_cancer(return_X_y=True)
```

Formater les données

```
# Importer le découpage
from sklearn.model_selection import
    train_test_split
# Découper le jeu en train/test
X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.2)
# Importer la standardisation
from sklearn.preprocessing import StandardScaler
# Standardisation
X = StandardScaler().fit_transform(X)
```

K plus proches voisins

```
# Importer le modèle
from sklearn.neighbors import KNeighborsClassifier
# Initialiser le modèle
model = KNeighborsClassifier(n_neighbors=3)
# Entraîner le modèle
model.fit(X_train, y_train)
# Calculer le score d'exactitude du modèle
model.score(X_test, y_test)
```

Arbres de décision

```
# Importer le modèle
from sklearn.tree import DecisionTreeClassifier
# Initialiser le modèle
model = DecisionTreeClassifier()
# Entraîner le modèle
model.fit(X_train, y_train)
# Calculer le score d'exactitude du modèle
model.score(X_test, y_test)
```

K-Moyennes

```
# Importer le modèle
from sklearn.cluster import KMeans
# Initialiser le modèle
model = KMeans(n_clusters=K)
```

```
# Entraîner le modèle
model.fit(X)
# Accéder aux classes formées
model.labels_
# Accéder à l'inertie
model.inertia_
```

CAH

```
# Importer le modèle
from sklearn.cluster
    import AgglomerativeClustering
# Initialiser le modèle
model = AgglomerativeClustering
    (distance_threshold=0, n_clusters=None)
# Entraîner le modèle
model.fit(X)
```

Affichage

```
# Importer pyplot
import matplotlib.pyplot as plt
# Importer le modèle d'arbres
from sklearn import tree
# Affichage textuel de l'arbre
text_representation = tree.export_text(model)
print(text_representation)
# Affichage graphique de l'arbre
fig = plt.figure(figsize=(25,20))
_ = tree.plot_tree(model,
    feature_names=["v1", "v2", "v3", "v4"],
    class_names=["c1", "c2", "c3"],
    filled=True)
fig.savefig("decistion_tree.png")

# Afficher un nuage de points avec classes
colors = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
c = [colors[y[i]] for i in range(y.shape[0])]

plt.scatter(X[:, 0], X[:, 1], c=c)
plt.show()
# Pour des classes prédites remplacer
c = [colors[model.labels_[i]] for i
    in range(model.labels_.shape[0])]

# Afficher la courbe de perte d'inertie
K = [i for i in range(2, N)]
plt.plot(K, inertias)
plt.show()
```

Pandas

```
# Importer numpy
import pandas as pd
# Importer un jeu de données
df = pd.read_csv("pokemon.csv", delimiter=",")
# Sélectionner les N premières colonnes
df = df[df.columns[:N]]
# Afficher la variable "name" du i-ème élément
print("Nom :", df.loc[i, "name"])
# Retourne les indices où la valeur de liste i
np.where(liste == i)[0]
# Selectionne les colonnes selon leur nom
X = df[["variable_1", "variable_2", "variable_2"]]
```

Dendrogramme

```
# Importer le dendrogram
from scipy.cluster.hierarchy import dendrogram

# Définir la méthode
def plot_dendrogram(model, **kwargs):
    counts = np.zeros(model.children_.shape[0])
    n_samples = len(model.labels_)
    for i, merge in enumerate(model.children_):
        current_count = 0
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1
            else:
                current_count += counts[child_idx-n_samples]
        counts[i] = current_count

    linkage_matrix = np.column_stack(
        [model.children_, model.distances_, counts]
    ).astype(float)

    dendrogram(linkage_matrix, **kwargs)

# Afficher le dendrogramme
plot_dendrogram(model, truncate_mode="level", p=3)
plt.xlabel("Nombre de points dans la classe
    (ou index sans parenthèses).")
plt.show()
```

Pour le reste, rendez vous sur la docummentation!